Red Social Pascualina

Implementación del Modelo Físico y Script DDL

Tarea 3 - Bases de Datos 1

Integrantes del Grupo 6:

Eisson Javier Oyola Tique
Julian Quintero Copete
Jonathan Andres Serna Velasco
Daniel Alejandro Stuart Muñoz

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas
Medellín, Colombia

13 de octubre de 2026

Índice

1. Introducción **2**

2. Selección y Justificación del SGBD **2**
2.1. Justificación de la Elección de PostgreSQL 2
2.2. Comparativa Detallada de Motores Considerados 2

3. Transformación del Modelo Lógico al Modelo Físico **2**

4. Diccionario de Datos Físico **3**
4.1. Tabla: Usuarios 3
4.2. Tabla: Perfiles 3
4.3. Tabla: Publicaciones 3
4.4. Tabla: Grupos 4
4.5. Tabla Intermedia: Usuarios_Grupos 4
4.6. Tabla: Conexiones (Relación Reflexiva N:M) 5
4.7. Tabla: Eventos 5
4.8. Tabla: Comentarios 5

5. Implementación de Relaciones y Acciones Referenciales **5**

6. Estructura y Explicación del Script DDL **5**
6.1. Secuencia de Ejecución 6
6.2. Fragmento del Script DDL en PostgreSQL 6

7. Conclusiones Individuales y Reflexión Crítica **7**
7.1. Eisson Javier Oyola Tique 7
7.2. Julian Quintero Copete 7
7.3. Jonathan Andres Serna Velasco 7
7.4. Daniel Alejandro Stuart Muñoz 7

1. Introducción

El presente informe expone el proceso de diseño e implementación del **modelo físico de base de datos** para la **Red Social Estudiantil Pascualina**. Tras haber definido el modelo conceptual y haber estructurado y normalizado el modelo lógico hasta la Tercera Forma Normal (3FN) en fases previas, esta etapa centra su alcance en la materialización funcional de la base de datos dentro de un Sistema de Gestión de Bases de Datos (SGBD) relacional real.

El objetivo principal es traducir la arquitectura de tablas, atributos y relaciones a un script DDL (*Data Definition Language*) riguroso. Este script garantiza la integridad referencial, la optimización del almacenamiento mediante la selección precisa de tipos y tamaños de datos, y el establecimiento de restricciones de clave primarias, clave foráneas y reglas de modificación/eliminación en cascada.

2. Selección y Justificación del SGBD

Para el soporte de la Red Social Pascualina se ha seleccionado **PostgreSQL** como el motor de base de datos principal. A continuación se detallan las razones técnicas de su elección y la comparativa exhaustiva frente a dos alternativas representativas en la industria.

2.1. Justificación de la Elección de PostgreSQL

PostgreSQL es un SGBD relacional orientado a objetos de código abierto, reconocido por su estricto cumplimiento de las propiedades ACID (*Atomicity, Consistency, Isolation, Durability*) y el estándar ANSI SQL. Sus principales ventajas para este proyecto radican en:

Soporte Nivel Enterprise y Gratuito: Licencia de código abierto sin costos de licenciamiento, ideal para proyectos institucionales y universitarios.

Extensibilidad y Tipos Avanzados: Manejo nativo de secuencias auto-incrementales mediante el tipo `SERIAL`, fechas con zonas horarias y capacidad nativa para consultar e indexar datos semiestructurados (como `JSONB`), lo cual facilitará la transición hacia la Tarea 4 (Big Data / IoT).

Rigurosidad en Integridad Referencial: Control estricto en la declaración de claves foráneas y acciones de borrado/actualización en cascada.

2.2. Comparativa Detallada de Motores Considerados

Conclusión de la selección: Aunque MySQL ofrece ligereza inicial y SQL Server posee herramientas GUI consolidadas, PostgreSQL representa la opción más equilibrada entre costo cero, robustez en la integridad relacional y versatilidad para escalar hacia arquitecturas híbridas (SQL + NoSQL).

3. Transformación del Modelo Lógico al Modelo Físico

La conversión del modelo lógico al modelo físico requirió traducir los conceptos abstractos a objetos reales de base de datos aplicando las siguientes reglas:

Tabla 1: Matriz comparativa de SGBD considerados.

Criterio	PostgreSQL (Elegido)	MySQL	SQL Server
Licencia y Costo	Código abierto (Gratuito, PostgreSQL License).	Código abierto (GPL / Comercial por Oracle).	Propietaria (Licenciamiento costoso).
Cumplimiento SQL	Cumplimiento casi total del estándar ANSI SQL.	Cumplimiento parcial (variaciones en sintaxis y restricciones).	Extensión Dialecto T-SQL rigurosa.
Soporte JSON / NoSQL	Excelente soporte nativo e indexación con `JSONB`.	Soporte básico mediante tipo `JSON`.	Soporte JSON mediante funciones relacionales.
Integridad y Triggers	Altamente robusto, comprobaciones estrictas.	Susceptible a modos permisivos según configuración.	Muy robusto pero dependiente del ecosistema Microsoft.
Facilidad de uso GUI	Excelente soporte con **pgAdmin 4**.	Integración con MySQL Workbench.	Integración con SSMS.

1. **Entidades Físicas:** Cada entidad lógica se transformó en una estructura `CREATE TABLE`.
2. **Claves Primarias (PK):** Se implementó el tipo sintético auto-incremental `SERIAL` en PostgreSQL para generar claves primarias numéricas enteras y eficientes en índices.
3. **Relaciones 1:1:** La relación entre `Usuarios` y `Perfiles` se implementó mediante una clave foránea (`id_usuario`) en la tabla `Perfiles` acompañada de la restricción `UNIQUE`.
4. **Relaciones 1:N:** Se incluyeron claves foráneas en la tabla hija (ejemplo: `id_usuario` en `Publicaciones` o `id_grupo` en `Eventos`).
5. **Relaciones N:M:** Se transformaron en tablas intermedias de unión con claves primarias compuestas formadas por las claves de las entidades participantes (ejemplo: `Usuarios_Grupos`, `Usuarios_Intereses`, `Conexiones`).
6. **Estandarización de Nomenclatura:** Todos los nombres de tablas y campos adoptaron la convención *PascalCase* para tablas y *snake_case* para columnas y restricciones (`fk_...`).

4. Diccionario de Datos Físico

A continuación se detalla la estructura física de cada tabla implementada en PostgreSQL, con la correspondiente justificación de la selección de tipos de datos:

4.1. Tabla: Usuarios

4.2. Tabla: Perfiles

4.3. Tabla: Publicaciones

Campo	Tipo Dato	Llave	Justificación Técnica
id_usuario	SERIAL	PK	Entero auto-incremental eficiente para indización.
nombre	VARCHAR(100)	NOT NULL	Permite longitud variable optimizando espacio de memoria.
correo	VARCHAR(150)	UNIQUE, NN	Longitud suficiente para dominios institucionales largos.
carrera	VARCHAR(100)	NULL	Texto de longitud variable para nombres de programas académicos.
fecha_registro	TIMESTAMP	DEFAULT	Registra la marca de tiempo exacta del registro del usuario.

Campo	Tipo Dato	Llave	Justificación Técnica
id_perfil	SERIAL	PK	Identificador único interno.
id_usuario	INT	FK, UNIQUE	Enlace 1:1 con Usuarios. `UNIQUE` impide múltiples perfiles.
biografia	TEXT	NULL	Permite descripciones largas sin límite rígido de caracteres.

Campo	Tipo Dato	Llave	Justificación Técnica
id_publicacion	SERIAL	PK	Clave primaria auto-incremental.
contenido	TEXT	NOT NULL	Adecuado para publicaciones de longitud variable (memes, dudas, noticias).
fecha	TIMESTAMP	DEFAULT	Marca cronológica exacta de publicación.
id_usuario	INT	FK	Identifica al estudiante autor de la publicación.

4.4. Tabla: Grupos

Campo	Tipo Dato	Llave	Justificación Técnica
id_grupo	SERIAL	PK	Identificador único del grupo o club.
nombre	VARCHAR(100)	NOT NULL	Almacena el nombre del equipo de estudio o club.
tematica	VARCHAR(50)	NULL	Categoría corta (ej. Hackatón, Programación).
fecha_creacion	DATE	DEFAULT	Guarda la fecha (día, mes, año) de apertura del grupo.

4.5. Tabla Intermedia: Usuarios_Grupos

Campo	Tipo Dato	Llave	Justificación Técnica
id_usuario	INT	PK, FK	Parte 1 de la clave primaria compuesta. FK a Usuarios.
id_grupo	INT	PK, FK	Parte 2 de la clave primaria compuesta. FK a Grupos.
fecha_union	DATE	DEFAULT	Registra la fecha de ingreso al grupo.
rol	VARCHAR(30)	DEFAULT	Define la función (*'Líder'*, *'Miembro'*).

4.6. Tabla: Conexiones (Relación Reflexiva N:M)

Campo	Tipo Dato	Llave	Justificación Técnica
id_usuario	INT	PK, FK	Usuario seguidor. FK a Usuarios.
id_usuario_seguido	INT	PK, FK	Usuario seguido. FK a Usuarios (Red reflexiva).
fecha_conexion	DATE	DEFAULT	Almacena la fecha de inicio del seguimiento.

4.7. Tabla: Eventos

Campo	Tipo Dato	Llave	Justificación Técnica
id_evento	SERIAL	PK	Identificador único del evento.
titulo	VARCHAR(150)	NOT NULL	Nombre o enunciado principal de la actividad.
descripcion	TEXT	NULL	Detalles explicativos amplios de la agenda.
fecha	DATE	NOT NULL	Día de realización programado.
lugar	VARCHAR(100)	NULL	Ubicación física o enlace a reunión virtual.
id_grupo	INT	FK (Nullable)	Grupo organizador. Si es NULL, es un evento individual.

4.8. Tabla: Comentarios

Campo	Tipo Dato	Llave	Justificación Técnica
id_comentario	SERIAL	PK	Clave primaria auto-incremental.
contenido	TEXT	NOT NULL	Texto del comentario vertido en un post.
fecha	TIMESTAMP	DEFAULT	Estampa cronológica del comentario.
id_usuario	INT	FK	Autor del comentario.
id_publicacion	INT	FK	Post receptor del comentario.

5. Implementación de Relaciones y Acciones Referenciales

Para salvaguardar la coherencia de los datos en escenarios de borrado o actualización de entidades padre, se configuraron reglas referenciales específicas:

6. Estructura y Explicación del Script DDL

El script `script_redsocial.sql` fue diseñado garantizando la idempotencia y el orden de ejecución estricto según la jerarquía de dependencias.

Tabla 2: Matriz de relaciones y reglas referenciales.

Tabla Padre	Tabla Hija	Acción On Delete	Justificación Funcional
Usuarios	Perfiles	CASCADE	Si un usuario se elimina, su perfil debe desaparecer automáticamente.
Usuarios	Publicaciones	CASCADE	Borra en cascada los contenidos generados por la cuenta eliminada.
Usuarios	Comentarios	CASCADE	Mantiene limpios los comentarios de cuentas cerradas.
Publicaciones	Comentarios	CASCADE	Al borrar una publicación, se eliminan automáticamente todos sus comentarios.
Grupos	Eventos	SET NULL	Si un grupo se disuelve, el evento persiste como actividad comunitaria independiente.
Usuarios / Grupos	Usuarios_Grupos	CASCADE	Si se elimina un usuario o grupo, se remueve la relación de membresía.
Usuarios	Conexiones	CASCADE	Elimina los vínculos de seguimiento asociados al usuario borrado.

6.1. Secuencia de Ejecución

1. **Limpieza de Entorno (`DROP TABLE IF EXISTS ... CASCADE`):** Permite la reejecución sin conflictos por tablas previamente existentes.
2. **Entidades Independientes (Padre):** Creación de `Usuarios`, `Grupos`, `Intereses` y `Habilidades`.
3. **Entidades Dependientes (Hijas 1:1 y 1:N):** Creación de `Perfiles`, `Publicaciones`, `Eventos` y `Comentarios`.
4. **Tablas de Relación N:M (Intermedias):** Creación de `Usuarios_Grupos`, `Conexiones`, `Usuarios_Intereses`, `Usuarios_Habilidades` y `Asistentes_Eventos`.

6.2. Fragmento del Script DDL en PostgreSQL

```
-- Tabla Perfiles con relacion 1:1 y borrado en cascada
CREATE TABLE Perfiles (
    id_perfil SERIAL PRIMARY KEY,
    id_usuario INT NOT NULL UNIQUE,
    biografia TEXT,
    CONSTRAINT fk_perfiles_usuarios FOREIGN KEY (id_usuario)
        REFERENCES Usuarios(id_usuario)
        ON DELETE CASCADE
        ON UPDATE CASCADE
```

```
);
```

7. Conclusiones Individuales y Reflexión Crítica

7.1. Eisson Javier Oyola Tique

La implementación física en PostgreSQL representó una oportunidad fundamental para comprender el paso de la teoría conceptual al estándar DDL en producción. Configurar acciones referenciales como `ON DELETE CASCADE` frente a `ON DELETE SET NULL` me permitió analizar el impacto que tienen las restricciones en la preservación de la memoria histórica del sistema.

7.2. Julian Quintero Copete

Durante esta fase, la mayor dificultad técnica consistió en secuenciar correctamente las sentencias DDL para prevenir errores por dependencias circulares o violaciones de llaves foráneas no creadas aún. Este ejercicio me demostró el valor de la planificación estricta en el desarrollo de bases de datos relacionales.

7.3. Jonathan Andres Serna Velasco

Elegir entre diferentes tipos de datos, como definir campos de longitud fija contra tipos dinámicos (`VARCHAR` vs. `TEXT`), amplió mi criterio técnico respecto al equilibrio entre la optimización del espacio de almacenamiento físico y el rendimiento de las búsquedas en el motor.

7.4. Daniel Alejandro Stuart Muñoz

El análisis comparativo entre PostgreSQL, MySQL y SQL Server me ayudó a vislumbrar cómo las decisiones arquitectónicas influyen en el ciclo de vida del software. PostgreSQL demostró ser la alternativa más flexible, sobre todo considerando el soporte nativo de tipos de datos complejos para las fases posteriores del proyecto.